

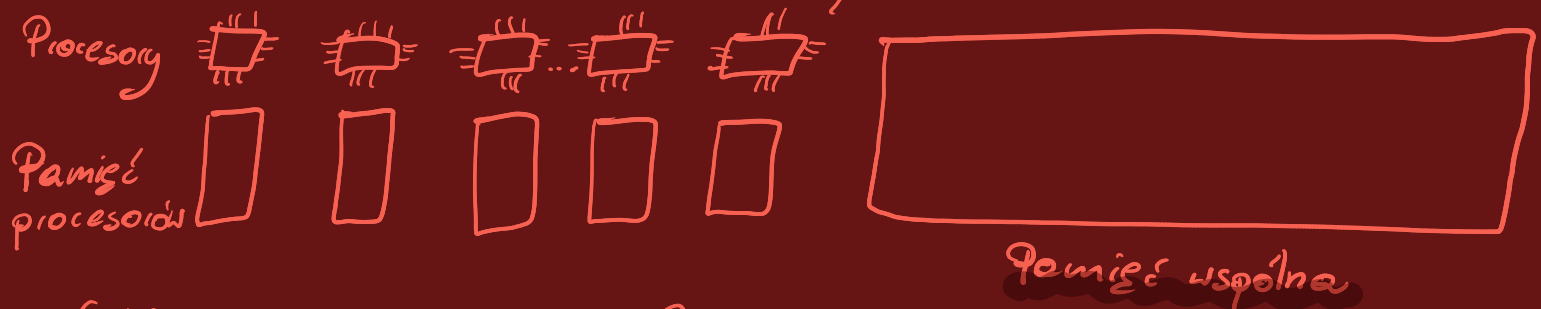
AiSD

Wykład XXVIII

Algorytmy równoległe

Model: **PRAM** - parallel RAM machine

W modelu tym mamy:



Cykl maszyny:

- ~ odczyt
- ~ obliczenia
- ~ zapis

Problemy:

- ~ 2 procesory mogą nadpisać tę samą wartość.

Minimalizujemy liczbę cykli

Tryby odczytu i zapisu

CREW (Concurrent Read Exclusive Write)

- C** - concurrent (może być przez wiele procesorów)
- E** - Exclusive (wyłączanie przez 1 procesor)
- R** - Read

CRAW (Concurrent Read Atomic Write)

- C** - concurrent
- E** - Exclusive
- W** - Write

Możliwość algorytmów:

1. Optymalna kolejność mnożenia macierzy:

$$\Theta\left(\frac{n^3}{n}\right) = \Theta(n^2)$$

↳ n procesorów wykonuje pracę

2. Sortowanie - co jeśli n procesorów wykonuje logarytmiczną pracę?

W świecie algorytmów równoległych istnieje odpowiednik:
 $P \subseteq RP$ - czyli mamy klasy problemów, które trudno u równoleglic, których u równoleglenie implikuje u równoleglenie innych.

Niestety maszyny PRAM nie udało się skonstruować do dziś :-C

Algorytmy rozproszone - mamy procesory rozproszone komunikujące się falami radiowymi (!)

Sieci komparatorów

- druty
- komparatory



Problem:

Dane: n

Zadanie: Skonstruować sieć komparatorów sortuje każdy ciąg wejściowy długości n . Taką sieć nazywamy siecią sortującą.

Lemat 1 - Sieć komparatorów jest sortująca o n wejściach iff porządkuje każdy ciąg $10, 10^n$

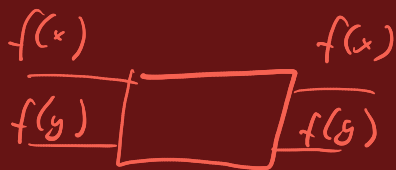
Lemat 2

Jeśli sieć komparatorów ciąg wejściowy $\langle a_1, \dots, a_n \rangle$ przekształca w $\langle b_1, \dots, b_n \rangle$ ($b_i b_i^n$ jest permutacją $10, 10^n$) to dla dowolnej funkcji niemalejącej ta sieć przekształca $\langle f(a_1), \dots, f(a_n) \rangle$ w $\langle f(b_1), \dots, f(b_n) \rangle$

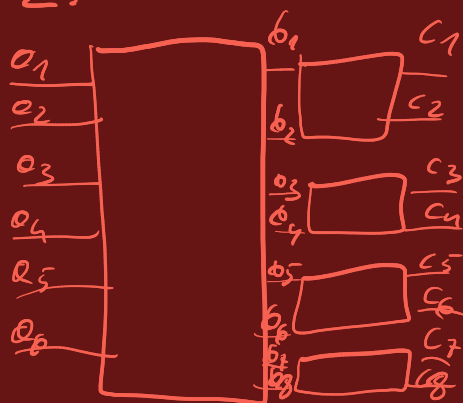
d-d:

$n=2$

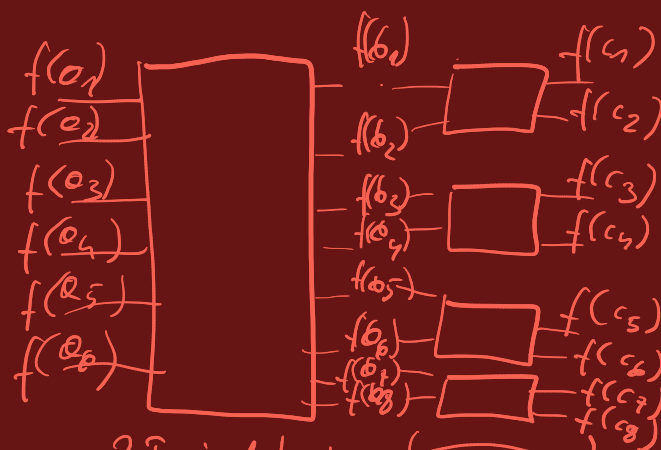
Niech $x < y$. Skoro f jest niemalejąca to $f(x) \leq f(y)$



$n > 2$:



$\Rightarrow$



Zat. indukcyjne z tego, że f jest niemalejąca.

$$\Rightarrow c_i = \min(b_i, b_{i+1}) \Rightarrow f(c_i) = \min(f(b_i), f(b_{i+1}))$$

$$\Rightarrow c_{i+1} = \max(b_i, b_{i+1}) \Rightarrow f(c_{i+1}) = \max(f(b_i), f(b_{i+1}))$$

d-d (Lematu 1):

$\Rightarrow$ Orazuiste

$\Leftarrow$ Przez kontabeggie

Zatowimy, że $\exists$ ciąg $\{a_i\}_{i=1}^n$ którego niesortuje.

Oznacza to, że $\exists a_i, a_j$ $a_i > a_j$, ale na wyjściu a_i będzie wcześniej.

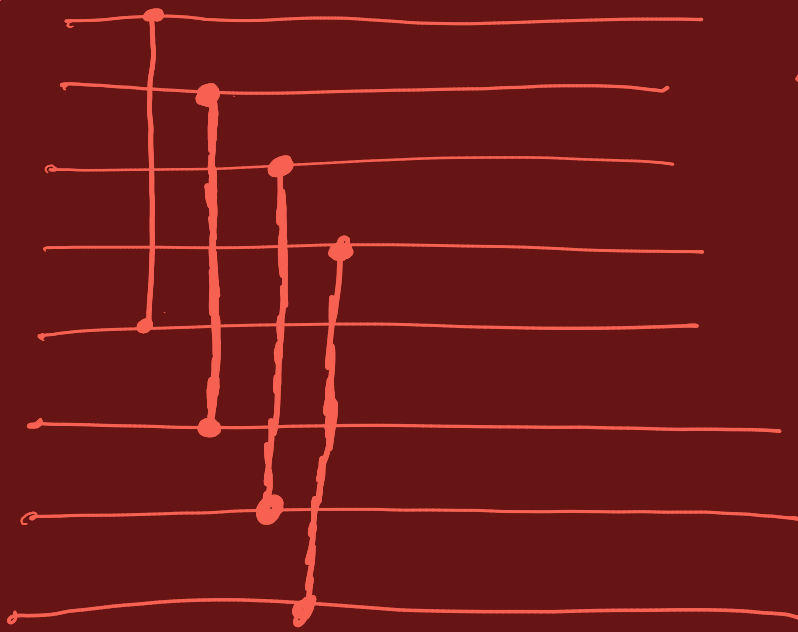
$$\text{Definiujemy } f(x) = \begin{cases} 0 & \text{if } x \leq a_j \\ 1 & \text{if } x > a_i \end{cases}$$

Z lematu 2 i z faktu, że f jest niemalejąca $f(a_i) = 1$ pojawi się przed $f(a_j) = 0$, czyli taka sieć nie sortuje poprawnie $\{f(a_k)\}_{k=1}^n$.

Sieć sortująca

1. Sieć półczyszcząca - łączymy druty i oraz $\frac{n}{2} + i$

$n=8$



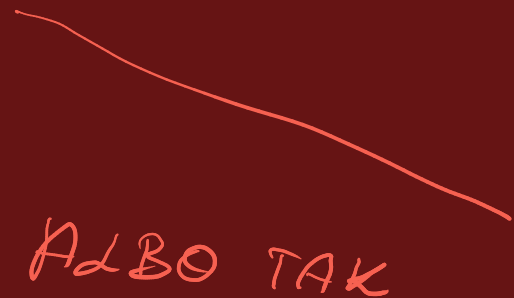
Fakt - jeżeli na
wejściu jest 10,10-owy
ciąg bitoniczny, to na
wyjściu:

- obie połowy są bitoniczne
- każdy element z górnej
połowy $\leq$ każdy element
z dolnej połowy
- górna lub dolna
połowa jest czysta (tj.
złożona z samych 0 lub 1)

Def. ciąg bitoniczny.

Wg K20:

→ Monotoniczny:

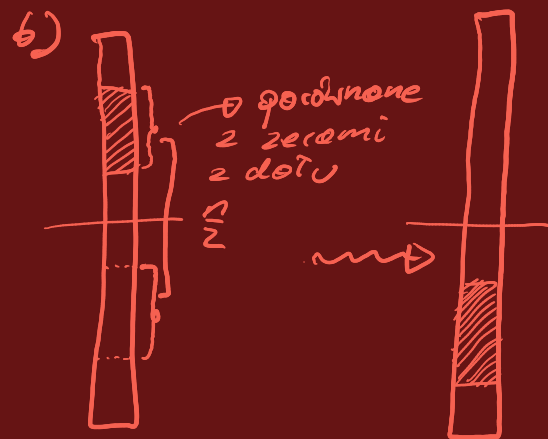
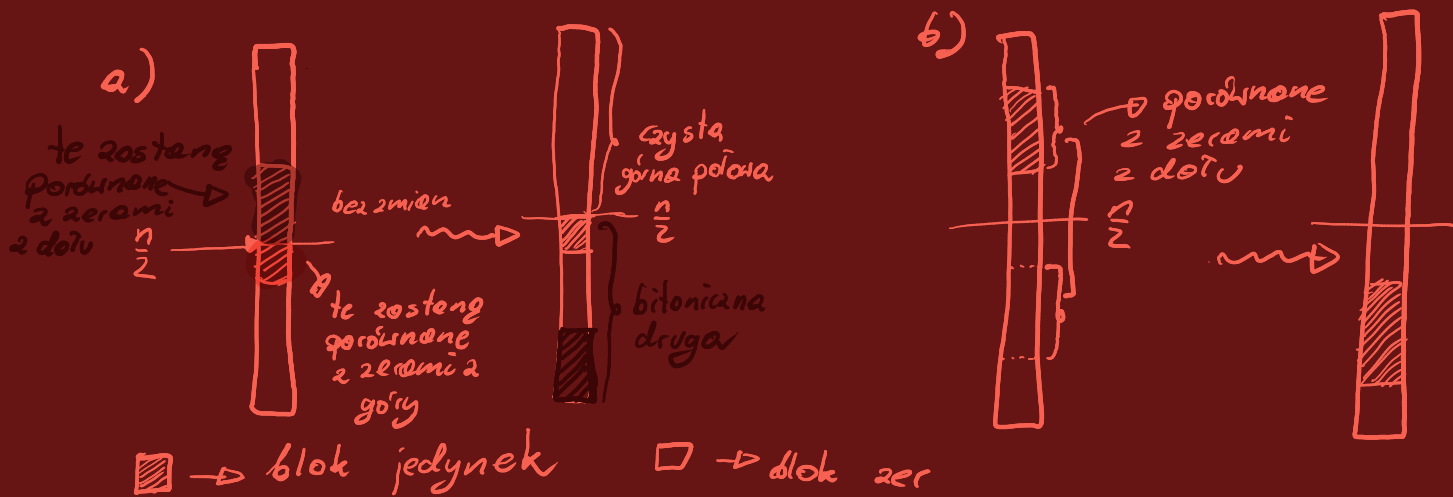


Binarne ciągi bitoniczne:

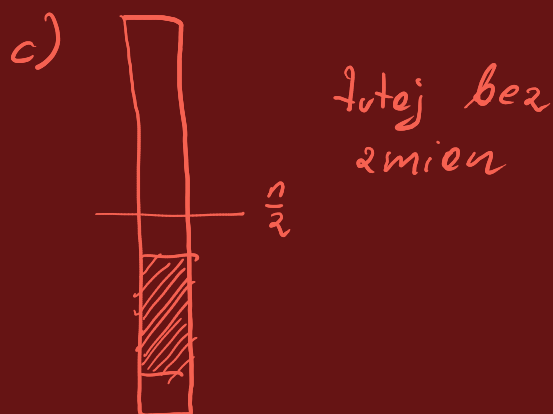
$$\rightarrow 0^i 1^j 0^k$$

$$\rightarrow 1^i 0^j 1^k$$

Żalaliśmy, że wejście jest postaci $O(n^2)$

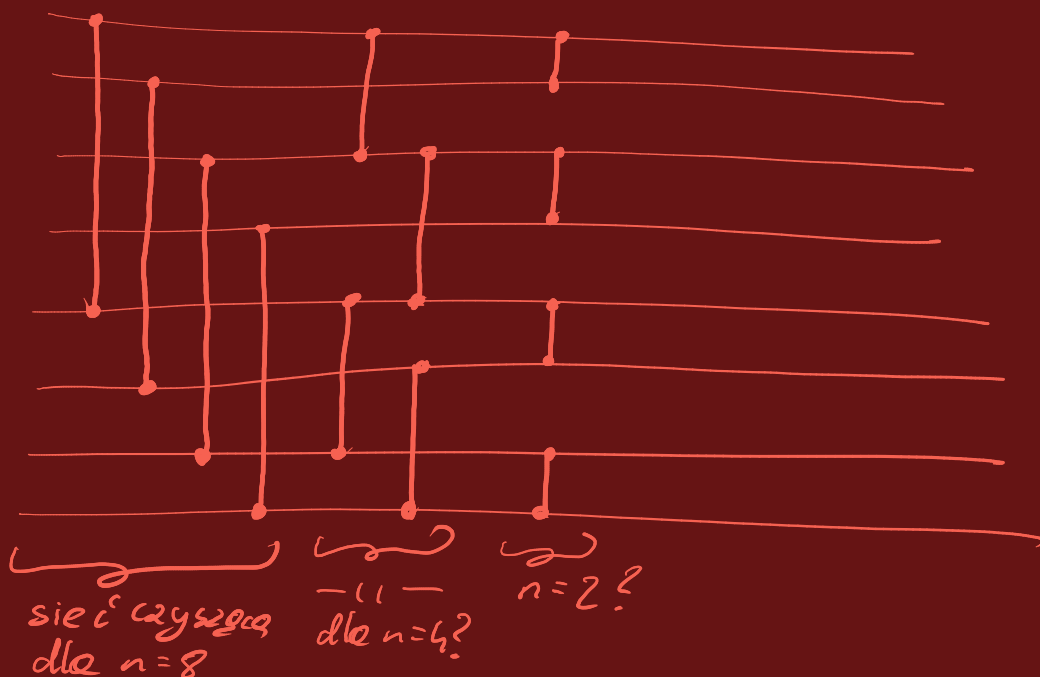


„Ładne wielkie $A_j - B_j$ - tu się nie dzieje” ~ KŁO

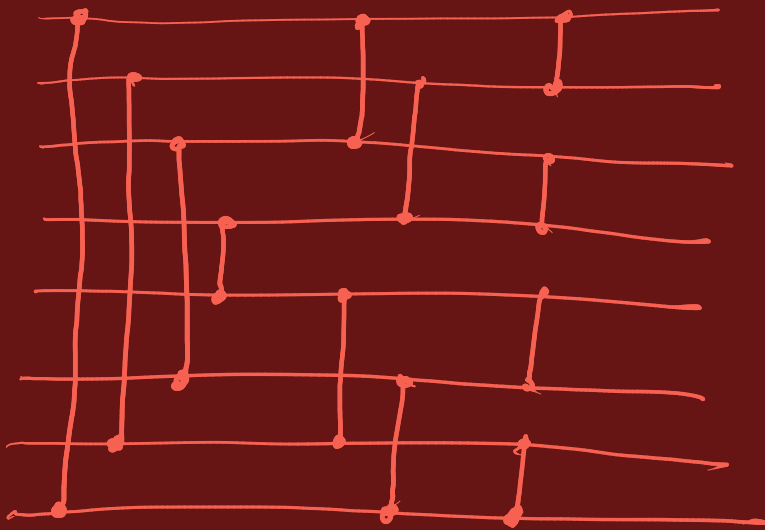


W a, b, c jedna połowa
stoje się czysta, wszędzie
mamy ciągi bitoniczne
(o ile stały ciąg jest bitoniczny...)
na c $\forall$ element z górnej
połowy jest $\leq$ $\forall$ element
z dolnej.

Sieć sortująca ciągi bitoniczne:



Sieć skalująca



Sieć sortująca



$$\text{Głębokość} = \log^2 n$$

Pytania

1. Czy istnieje płytsza? Tak

Ajtai, Komlós, Szemerédi

np $O(6000 \log n) \rightarrow$ nigdy nie doczekamy takich zastosowań $\sim KLO$

2. Dlaczego ostatnia notatka ma krwisty kolor?

~ „Bo krew to esencja płyn” ~ Johann Wolfgang von Goethe ~ Faust.1.